

Exercice — Intro Bootstrap : premier projet CDN

 **DURÉE** : ● 5 min | ● 15 min | ● 20 min
 **Niveau Bloom** : Appliquer (3) / Créer (6)
 **Prérequis** : Leçon XIV (Intro Bootstrap)

Objectif

Installer Bootstrap via CDN sur une page HTML5 vide et **utiliser** les classes utilitaires de base pour reproduire une mise en page responsive sans écrire de CSS custom.

● Échauffement

Créer un fichier `intro-bootstrap.html`. Inclure les CDN Bootstrap dans le squelette HTML :

- Dans le `<head>` : `<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">`
- Juste avant `</body>` : `<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>`

Puis dans le `<body>`, ajouter :

```
<div class="container py-5">
  <h1 class="text-primary mb-4">Hello Bootstrap !</h1>

  <p class="lead">Voici mon premier composant Bootstrap.</p>

  <button class="btn btn-primary">Bouton primary</button>
  <button class="btn btn-success">Bouton success</button>
  <button class="btn btn-outline-danger">Outline danger</button>
</div>
```

Ouvrir dans le navigateur. **Observer** :

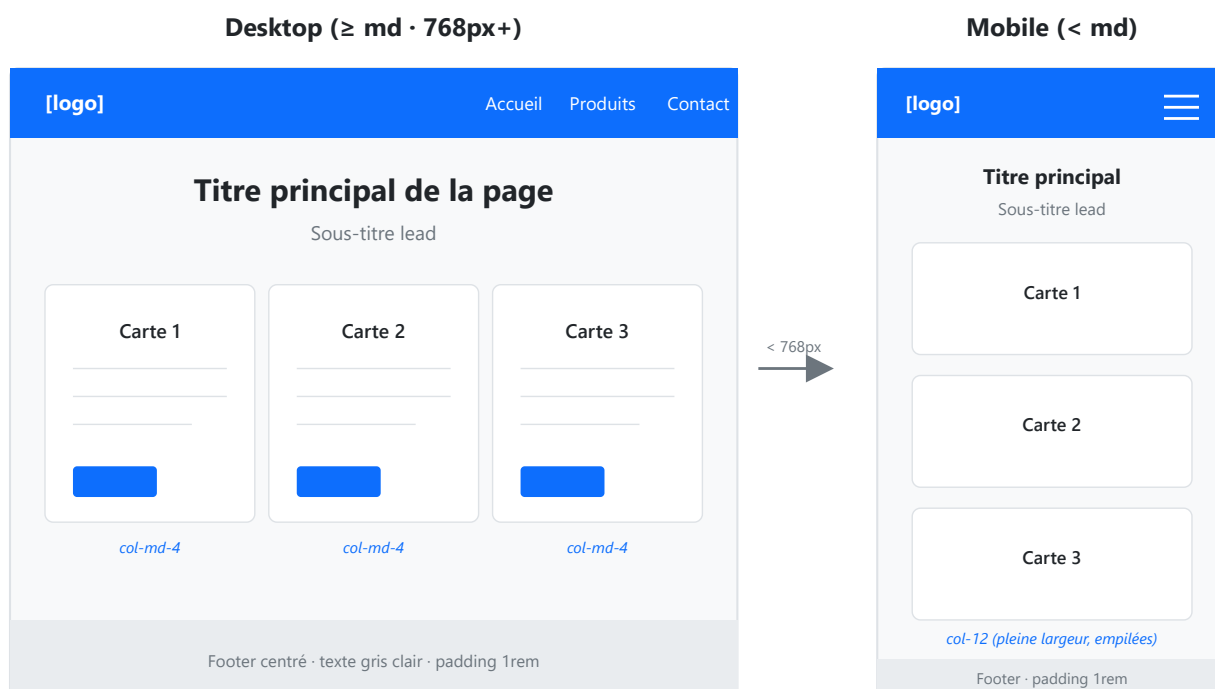
- Le titre est en bleu Bootstrap (couleur `primary` du framework).
- Les boutons ont des paddings, des arrondis et des couleurs cohérents avec le design system Bootstrap.
- Le `container` centre le contenu avec une largeur max responsive.
- `py-5` ajoute du padding vertical (`3rem` haut + bas).

Inspecter dans DevTools la classe `.btn-primary` : observer les nombreuses propriétés CSS qu'elle injecte. Sans Bootstrap, il faudrait écrire toutes ces règles à la main.

● Normal — À vous de jouer

Sans rien réutiliser du ●, créer `mes-composants.html` avec **uniquement** la classe utilitaire Bootstrap (pas de CSS custom) :

Objectif : reproduire visuellement la maquette suivante.



Contraintes :

- Utiliser `.container` ou `.container-fluid`.
- Utiliser `row` + `col-md-4` pour les 3 cards.
- Utiliser `.card`, `.card-body`, `.card-title`, `.card-text` pour chaque carte.
- Utiliser `.bg-primary`, `.text-white`, `.text-muted`, `.text-center`, `.py-*`, `.px-*`, `.my-*` au besoin.
- **Aucune balise `<style>`**, aucun CSS custom — uniquement des classes Bootstrap.

Indices :

- Pour le header : `<nav class="navbar navbar-expand-md bg-primary navbar-dark">`.
- Pour aligner les liens à droite : `ms-auto` dans le `<ul class="navbar-nav">`.
- Le `.card` n'a pas de hauteur fixée ; pour aligner 3 cards sur la même hauteur dans une row : ajouter `h-100` sur chaque card.

Bonus : ajouter une **icône Bootstrap** dans chaque bouton de carte. Inclure le CDN :

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.css" rel="stylesheet">
```

Puis `<i class="bi bi-arrow-right"></i>` dans un bouton.

Livrable : `mes-composants.html` rendu correct sur mobile et desktop, sans `<style>`.

● Défi — Bootstrap Icons + comportements avancés

Étendre `mes-composants.html` avec deux comportements avancés sans écrire de JS custom :

1. **Bouton retour en haut** : positionné fixed bas-droite, devient visible après scroll de 300px.
Indice : **Bootstrap n'a pas de classe pour ça**, mais une combinaison `position-fixed bottom-0 end-0 m-4` + une **simple ligne CSS** `body { scroll-behavior: smooth; }` + un `<a href="#top">` suffisent pour la fonction smooth-scroll. Le bouton fixed reste visible en permanence dans cette version (le toggle visibilité après 300px de scroll nécessiterait du JS — pas obligatoire ici).
2. **Tooltip Bootstrap** sur chaque icône : utiliser `data-bs-toggle="tooltip"` + initialisation JS minimaliste en bas de body :

```
<script>
  document.querySelectorAll('[data-bs-toggle="tooltip"]')
    .forEach(el => new bootstrap.Tooltip(el));
</script>
```

3. **Lighthouse** : viser **Performance ≥ 90** et **Accessibilité ≥ 95** sur la page.

Questions :

- Pourquoi Bootstrap pèse-t-il **~270 KB minifié** ? Que peut-on faire pour ne livrer que ce qui est utilisé (compilation Sass custom) ?
- Quelle alternative à CDN pour la prod ? (Indice : npm + Vite/Webpack).